GROUP INFORMATION	
Group Number	5
System Owned	PUP-SPOT
Group Members	Daniel Cueto Jenver Napenas Aaron Paul Imperial Emmanuel Manansala Justine Philip Raguin

INC-01 – Setup Script Fails to Adapt to Different Environments

Incident ID and Related Flaw: INC-01; QA-003 – Machine-Dependent and Inflexible Setup Scripts.

Affected Area: Windows setup scripts, PostgreSQL installation/service configuration, and database connection settings.

Affected Activity: Activity 3 – System Development, Database Setup, and Application Deployment.

Difficulty (Design Sheet): Easy; estimated 15–20 minutes (planned, not a measured duration).

Status: Confirmed source-code limitations; cross-machine startup failure has not been reproduced.

Incident Brief: On a machine with a different PostgreSQL installation directory or service name, the project's startup process may fail to initialize the expected database connection.

Scenario: A tester launches the application on a Windows computer where PostgreSQL is installed outside the predefined search locations or where its Windows service is named differently. The launchers attempt environment-specific recovery and may not establish a working database connection.

Evidence: QA-003 identifies fixed service references in setup_db.bat and setup_and_start.bat (postgresql-x64-18); predefined installation-path checks; different DB_NAME/DB_PORT launcher settings versus the DB_URL used by application.properties; and forceful port cleanup in start.bat and setup_and_start.bat. See the QA-003 source screenshots and recorded read-only rg output in the flaw document.

Expected Root Cause: Machine-dependent service/path assumptions and inconsistent configuration sources. This is the expected cause of the proposed scenario, not a proven cause of a live outage.

Recommended Fix: Document supported operating systems and a manual setup procedure; make PostgreSQL service/path settings configurable; consistently resolve the database connection URL; replace automatic termination of unrelated processes with safe port-conflict handling.

Verification Method: On an authorized test computer, configure PostgreSQL with a different service name or directory. Confirm connectivity, successful startup, and safe behavior when the application port is occupied.

Test Result / Limitation: The scripts were inspected without being run. No alternate-machine setup failure, PostgreSQL modification, or process termination was observed. Fault and reset scripts remain placeholders in the incident design sheet.

INC-02 – Lack of Separation Between Citizen and Administrative Application Functions

Incident ID and Related Flaws: INC-02; QA-007 – Missing Implementation of Application Role Separation; related QA-004 (missing API authentication) and QA-006 (unrestricted request modification).

Affected Area: Application-layer identity checks, citizen/admin APIs, documented role permissions, and interface access.

Affected Activity: Activity 4 – Application and Database Access-Control Review (drawing on Activity 3 roles documentation).

Difficulty (Design Sheet): Hard; estimated 25–30 minutes (planned).

Status: Role-enforcement gap confirmed by static source review; unauthorized runtime access not tested.

Incident Brief: Documented Citizen, Staff, Auditor, and Administrator responsibilities are not visibly enforced by the reviewed API endpoints. A citizen-facing client could potentially reach administrative operations if those endpoints are exposed.

Scenario: A user intended to access only citizen services attempts to manage administrator records or change document requests. The test checks whether authentication and role-based backend controls reject unauthorized requests.

Evidence: docs/user-roles.md distinguishes the application roles; AdminController.java exposes administrative CRUD endpoints, while CitizenController.java and DocumentRequestController.java expose other operations without visible identity/role checks. pom.xml does not list Spring Security. The presence of a shared index.html alone does not establish a security violation.

Expected Root Cause: Missing application-level authentication and role-based authorization enforcement; the documented role rules are not demonstrably applied to the inspected backend. This overlaps QA-004 and is not a separate proven exploit.

Recommended Fix: Implement authenticated user identities and backend authorization for each role and sensitive action; use record-level checks for citizen records and document requests; present role-appropriate interface controls without relying on the UI for security.

Verification Method: In an authorized local environment with synthetic records, exercise the APIs as Citizen, Staff, Auditor, and Administrator. Record positive and denied operations; confirm citizens cannot access admin-only functions and authorized administrators still can.

Test Result / Limitation: Source inspection supports the missing controls. No role-bypass request or exploitation test was performed. Fault and reset scripts remain placeholders in the incident design sheet.

INC-03 – No Automated Database Backup Procedure

Incident ID and Related Flaw: INC-03; QA-008 – No Documented Automated Database Backup Procedure.

Affected Area: Database backups, recovery procedures, scheduling, retention, and maintenance documentation.

Affected Activity: Activity 6 – Backup and Recovery (as assigned in the original incident design sheet); the QA report also references Activities 3 and 5.

Difficulty (Design Sheet): Medium; estimated 15–20 minutes (planned).

Status: Missing repository-level documentation confirmed; real backup availability and restore capability are unknown.

Incident Brief: The inspected repository does not explain how regular PostgreSQL backups are scheduled, retained, or restored. If the database is lost or corrupted, the team may not have a documented recovery path.

Scenario: After a simulated database outage, a tester seeks a recent usable backup and attempts to restore it into a separate non-production database. The scenario checks whether a reliable recovery procedure actually exists.

Evidence: QA-008 reports no identifiable backup/restore scripts or documented backup schedule, retention policy, or recovery instructions in the reviewed database directory, README, database documentation, progress documentation, and troubleshooting guide. No backup file or restore output was supplied.

Expected Root Cause: Undocumented or absent backup automation. The absence of repository instructions does not establish that no external backup exists.

Recommended Fix: Document and schedule consistent PostgreSQL backups (for example using pg_dump where appropriate), secure storage and retention rules, ownership/responsibility, and a step-by-step restore playbook.

Verification Method: With authorized test data, create a backup, restore to a separate test database, and compare expected tables and record counts. Record backup timestamps, backup location, retention behavior, and restore evidence without exposing private records or secrets.

Test Result / Limitation: Repository-level backup documentation gap is verified. No live backup/restore operation was carried out; actual restore readiness remains unverified. Fault and reset scripts were placeholders in the incident design sheet.

INC-04 – Missing Database Role Separation and Least-Privilege Permissions

Incident ID and Related Flaw: INC-04; no directly confirmed PostgreSQL privilege flaw in the current QA-001–QA-009 documentation. QA-004 and QA-007 relate to APPLICATION-layer authorization only.

Affected Area: PostgreSQL login roles, role memberships, schema/table privileges, and configured application runtime credentials.

Affected Activity: Activity 4 – Database Privileges and Role Management.

Difficulty (Design Sheet): Hard; estimated 30–40 minutes (planned).

Status: Open investigation; excessive database privileges have not been established by current evidence.

Incident Brief: Investigate whether the database runtime account has broader access than its job requires and whether database administration is properly separated from ordinary application activity.

Scenario: A database account used for routine application functions attempts an administrative database action. The investigation compares actual grants and permissions with the documented Activity 4 roles and expected least-privilege rules.

Evidence: The incident sheet identifies migration/setup SQL, application.properties, database configuration source, PostgreSQL role memberships, and grants as necessary sources. The flaw document establishes weak application API authorization, but it does not include a completed database privilege matrix or role-membership test proving a PostgreSQL grant violation.

Expected Root Cause: Potentially excessive runtime database permissions or missing separation of DBA and application accounts; this remains a hypothesis pending database verification.

Recommended Fix: Inventory actual database accounts, roles, default privileges, schema/table/sequence grants and current application credentials. Only after confirming excessive access, create or adjust least-privilege roles with an authorized administrator.

Verification Method: Use safe read-only catalog/privilege inspections and role-appropriate negative/positive tests on a test database. Verify intended operations succeed and unauthorized operations fail. Avoid exposing passwords or changing shared cluster permissions without authorization.

Test Result / Limitation: No verified database-level violation or corrective action is documented. The incident is an investigation scenario, not a confirmed occurrence. Fault/reset scripts remain placeholders.

INC-05 – Excessive Database Queries When Retrieving Document Requests

Incident ID and Related Flaw: INC-05; QA-009 – Excessive Database Queries When Retrieving Document Requests. (Corrected the source incident sheet's duplicate INC-04 label.)

Affected Area: GET /api/document-requests, DocumentRequestController, DocumentRequestService, DocumentRequestRepository, Hibernate entity loading.

Affected Activity: Activity 5 – Database Performance Monitoring and Query Optimization.

Difficulty (Design Sheet): Easy; estimated 5–10 minutes (planned).

Status: Manual reproduction reported in the QA document; full original Hibernate log sequence was not independently supplied.

Incident Brief: The document request endpoint loads related entities through multiple SQL statements when default findAll() results and lazy relationships are serialized, increasing database activity.

Scenario: As document requests grow, list retrieval repeatedly queries citizen, document-type, and administrator entities. The endpoint still returns records, but it may perform unnecessary SQL work.

Evidence: QA-009 identifies DocumentRequestService.getAllDocumentRequests() calling repository.findAll(), with Citizen, DocumentType, and Admin relationships marked FetchType.LAZY. Reported Hibernate logging found 11 SQL statements for the original path versus one for a custom JOIN FETCH implementation. The QA document includes illustrative log screenshots, but states that not all 11 original queries were captured.

Expected Root Cause: N+1 query behavior arising from default findAll() combined with lazy entity access during response serialization.

Recommended Fix: Use a targeted JOIN FETCH query, @EntityGraph, or DTO projection for required response fields; avoid fetching unrelated data; compare SQL counts, plans and response correctness before adopting an optimization.

Verification Method: Using a disposable local test setup, compare Hibernate SQL logging for the original and optimized GET endpoint, verify equivalent required response fields, and repeat on representative datasets. Preserve original records.

Test Result / Limitation: According to QA-009, original: 11 SQL statements; optimized: 1 statement (reported 90.9% reduction). Complete raw log output and performance under scale were not independently verified. The incident sheet states no fault/reset scripts are required; changes should be tested on a disposable copy.